

GIT | GITHUB

GIT

1. Famous FREE OPEN SOURCE VCS (Version Control System)
2. Used to keep track of every code version change, we can revert back specific version if there's a bug, when project gets big then it gets tough to manage code so companies use VCS, then most common used vcs is : git, piper etc.
- 3.
4. **INSTALLATION**
5. To use it, download git version from chrome - git scm.
6. In terminal, search for git or git -v, it will confirm if it's installed or not.
7. Now working, cheatsheet file for default git commands like setting your global username and email to git in terminal using : `git config --global user.name ""`
8. Why do : bcos whenever you make a commit, it shows from this person with username n email commit is coming.
9. Post this, we can start in our project, open in vs code, go to code directory.
10. Initialise git using : `git init`, now git will create a file says `.git` and will keep track of the project now.
11. Install git graph extension in vscode.
- 12.
13. **ADDING USING GIT**
14. Now, if your file in vscode coming with 'U' means untracked, git is not tracking.
15. To make it track use: `git add <filepth>`
16. Now post this, BOOM, git will track and show as 'A'.
17. Now whenever you write something in file it will show that line as green.
18. To check, added changes in console write : `git diff`, it will shows recent changes.
19. It's not possible to do git add for all files since in project, we have lot of files,so
20. We use : `git add .`
21. It adds all file of directory and git starts tracking all.
22. Also to remove file from tracking: `git rm <filepth>`
- 23.
24. **Git version changes as per time**
25. Use commit, everychange in code with time is called Commit.
26. Whenever we made change, we do commit.
27. Every commit has user friendly message, using: `git commit -m "First Commit"`
28. After successfully commit, you can see in code at what time which

- person added which code in code.
29. To check commit history, use: `git log`
 30. When we do change in code and remove earlier line or change line then, in `git diff` : earlier line comes as red and updated one as green.
 31. We always do : `git add .` , before committing, everytime first do `git add .` Then do `git commit -m ""`
 32. Refractor logs using: `git log --oneline`
 33. In detail view of logs using: `git show <commitID>`
 34. A very good command is line by line information of which person changed/added which code using : `git blame <filepath>`
 35. VCS usecase: we keep code and commits clean ,never commit complete code in one commit. Instead best is add one functionality then do commit n so on, best way.
 36. Using: `git status`: we can see after commit which files got modified/updated.
 37. When we modify files and try to commit then it will not since our **Staging Area** part is empty and we first need to keep our modified files into this staging file then we can do commit. That we can do using : `git add <filepath>`
 38. After: `git add <filepath>`, updated files comes to **Staging Area** then we do commit. Files in staging area means they are ready for commit . If we do: `git add .` , then it means get all files in staging area and we'll do commit of all.
 - 39.
 40. **HOW TO REVERT BACK?**
 41. What is Linked List ->  ->  -> 
 42. Means one node is connected to other and current node is said as HEAD and previous is TAIL.
 43. So `git` commits history is also like Linked List. So Head will be pointing to our latest commit.
 44. Two ways to revert back: First HARD Approach : `git reset --hard <commitId>` till which we want to go. So what it will do, if will point the head to this ID and the commit earlier which was after this commit will get lost/erase, so we use it but in some cases not. Like :
 45. I want to only remove specific code of commit bw other commits , if we go by hard way marking head point to earlier one than our current commit will be gone so here what we do is:
 46. Use: `git revert <commitId>` , it will only revert changes of that specific commit id without breaking the flow and and it will make a new commit head pointing where revert changes of that commit will be present.
 47. Git collaboration : let's say two person working on same project so how they both will be in sync with code sharing . Here we have common servers where every individual do commit of code (push) and others do pull of code and with time it keeps on creating commit History and track so that everyone will be in sync with code.

48. In real life we have many git common servers like GITHUB(free), GITLAB(paid), BITBUCKET(paid).

GITHUB

1. It's a GIT SERVER : Central Server where developers push n pull their code. Git is version control system used to track code and GITHUB (Single Source Of Truth) is git central/remoteyserver.
2. Okay so if we don't have github, then we use : git init , it makes repository in our local machine.
3. But we are using github, then : we can make directly in github.
4. Now post this, in our code we need to tell Git that it need to point to this GITHUB server and it need to keep our code sync with this server repo.
5. Using : git remote -v , firstly will give nothing. Bcos we haven't added remote till now. How to add?
6. Using : git remote add origin <repo path>
7. Here origin is bydefault name, we can give any but bydefault syntax name is origin only then we give path of our repo. What it will do? It will add remote of this repo.
8. Now if we run : git remote -v , it will show the server name , path and what we can do that is push n pull.
9. How to push code changes into github?using: git push -u origin main .
10. Here -u means upstream, main : these 2 are related to branch . Origin means remote server name.
11. Yes we can have multiple remote server as well, mostly used during open source contributions.
12. Now Main Question? Anyone can commit to our remote by just giving repo path?
13. No we need AUTHENTICATION and we have Secure Socket Shell (SSH Key)
14. So, we have private n public SSH KEYS. which we set in our local system and those who have access to that SSH key will be able to commit to remote server/repo.
15. How to add our SSH, for mac, get command from documentation of github and in terminal paste follow steps, and then in .ssh file copy public key ssh generated in console after cat and public id and in github create ssh key and paste here key and title it.
16. Now let's say you did a commit from git by making some change in code and check it using first: git status then, git diff then, git add . Then, git commit -m "message" then last: git push.
17. Now in github, you will see increase in commits count and our newly added commit also.
18. Now if you want to revert back so, in git do by hard way using: git reset --hard <commitId> and then: git push -f.

19. Git push -f means -f says forcefully change head in github to old one.

20.

21.

22. **BRANCHING IN GITHUB**

23. Branch is like railway track of commits.

24. Why need branching? : 2 persons are working on a project, and they both are committing to one branch only, so what will happen?: after some time, it gets full, branch will get pollute and will not be easy to track history. And let's say if you want to revert back other persons code then you need to either do hard reset or revert back one by one.

25. So to prevent this, we use branching - whenever we work on project, we always make our separate branch and always commit to that to prevent polluting of our main single branch.

26. When let's say other person is done with his work in his own branch and gives final confirmation to add his code into final branch, then what will happen let's say he made 3 commits so when we do final add his all commits will get combined into 1 single commit and will be added to our main branch. Now it makes history easier also called as merging., if we want to remove the added code, then we just need to revert the single commit of his.

27. That's the use case of Branching in Github.

28. Now working:

29. To check our current branch: git branch

30. Make new branch using: git branch <branchname>

31. Keep in mind, it's all in our local machine till now, so next, how to switch to new branch,

32. Using: git checkout branchname

33. Okay after doing some changes, then doing git add, then git commit, then git push, here it will show error as not set as upstream so copy paste shown command.

34. Now it will push and in github, you'll see a new branch just push code. So now our code from local and branch also will be into our github also.

35.

36. Now how to **merge** new branch code to our main, 2 ways: by CLI and by github ui.

37. First way, CLI - coming to main branch first, git checkout main, then, merging using : git merge origin/branchname.

38. Here origin is remote server name and from this server merge the branch with this branchname.

39. And push to our main: git push

40. Second way, UI github: after completing merging from ui, we need to update our local code also how, since in github it's updated so we need to come to main branch using : git checkout main then, use: git pull

41. Now it will bring the updated code after merging.

- 42.
43. **BRANCH NAME SHOULD BE?**
44. Can be using feat, wip, bug,junk.
45. Examples:
46. Git branch "feat/add-skip-button"
47. Git branch "feat/delete-skip-button"
48. Git branch "bug/login-not-working"
49. Git branch "wip/working"
50. Git branch "junk/test-no-use"
51. Wip(work in progress).
- 52.
53. Also one shortcut command: as we do:
54. Git branch "branchname" (creating)
55. Git checkout "branchname" (going)
56. Instead, do this-
57. Git checkout -b "branchname"
58. Single command it create and goes inside itself.
- 59.
60. **REBASE** : we do merge, there all commits get combined into 1 and we add it into our main trail and go on but it loses history of another branch commits as it makes all into 1 .
61. For this, we have rebase, where complete another branch commit trail comes forward to our main branch and then we move forward, now history is also maintained.
62. Command using: git rebase <branchname>
63. NOTE: Rebasing is done very rarely as it's make our commit history alot and pollute a lot. So we mostly use merge only.
- 64.
65. **STASHING (Temp Commit)-**
66. Let's say you want to pull new code as in main branch someone has updated the code and since you are already working in your code and you have some changes and u try to do: git pull but it will not work and say you unstage changes, but you want to pull so now two options- either do git add , git commit , git push but you are not done with your work you can't commit, so now, we have,
67. We use stashing using:
68. Git stash (will place our working code into some temp file)
69. Then do - git pull (updated code came)
70. Now, - git stash apply (will bring your working code back from temp file)
71. So now you have updated code with our inprogress code.
72. Mostly used case in companies.
73. HERE, END OF GIT N GITHUB BEGINNER TUTORIAL.

TOOLS PREFER FOR FUTURE PRACTISE:

1. GIT CHEATSHEET chrome
2. GIT VISUALIZER chrome
3. VS CODE EXTEN (Git Graph, Git Lens)